Module 5 Week B — Applied Lab: Trees & Ensembles

Train decision trees and random forests on an imbalanced customer churn dataset. Use `class_weight="balanced"` honestly (as an operating-point tool, not a magic recall-improver). Evaluate with PR-AUC and calibration curves. **Most importantly: demonstrate concretely what tree-based models are capable of that linear models are not.**

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

After this lab, you will be able to:

- Train `DecisionTreeClassifier` and `RandomForestClassifier` in scikit-learn, tuning `max_depth`, `n_estimators`, and `class_weight`
- Compute and interpret Expected Calibration Error (ECE), and explain why unbounded decision trees are poorly calibrated
- Extract and interpret random-forest feature importances
- Apply `class_weight="balanced"` and explain what changes (operating point at a fixed threshold) and what doesn't (PR-AUC ranking)
- Plot precision-recall curves and calibration curves using scikit-learn's `PrecisionRecallDisplay` and `CalibrationDisplay`
- Identify and structurally explain a specific customer where a random forest captures a pattern a logistic regression cannot

---

Setup

1. **Accept the assignment:** Accept the Lab 5B assignment
2. Clone your repo and open it in VS Code.
3. The repo contains `data/telecom_churn.csv` **— an expanded Petra Telecom customer sample** (~4,500 rows, 14 columns, `churned` target at ~16% positive rate). This is a different dataset from the drill: the drill used a smaller slice; the lab uses the full sample with richer signal in the primary features (tenure, contract length, support calls, monthly charges) and realistic noise in the rest.
4. Install dependencies:

```
pip install -r requirements.txt
```

5. Create your working branch:

```
git checkout -b lab-5b-tree-models
```

Tasks

Complete the 12 functions in `lab_trees.py`. The 7 tasks build on each other. Run your script with `python lab_trees.py` and run tests with `pytest tests/ -v`.

Task 1 — Load and split

Implement `load_and_split(filepath)`:

- Read the CSV at `filepath`
- Select `NUMERIC_FEATURES` into `X`
- Use `churned` as `y`
- Split 80/20 with `stratify=y` and `random_state=42`
- Return `(X_train, X_test, y_train, y_test)`

This is the same pattern as Week A. Stratification preserves the ~16% positive rate in both splits.

Task 2 — Decision tree and calibration comparison

Implement `build_decision_tree(X_train, y_train, max_depth=5, random_state=42)`:

- Train a `DecisionTreeClassifier` with the given parameters
- Return the fitted model

Then implement `compute_ece(y_true, y_prob, n_bins=10)`:

- Get the sort order of `y_prob` ascending: `order = np.argsort(y_prob)`. Reindex both arrays with it so that positions `0 ... n-1` are now in probability-ascending order.
- Split `np.arange(n)` into `n_bins` equal-count bins with `np.array_split` — this gives you the indices into your already-sorted arrays, so each bin contains contiguous samples at similar predicted-probability levels.
- For each bin, compute `|mean_predicted_prob − fraction_actual_positive|`, weighted by `bin_size / n`
- Return the weighted sum (the Expected Calibration Error)

Then implement `compare_dt_calibration(X_train, X_test, y_train, y_test)`:

- Train a decision tree with `max_depth=None` (unconstrained); compute its test-set ECE
- Train another with `max_depth=5`; compute its test-set ECE
- Return `{"ece_unbounded": ..., "ece_depth_5": ...}`

What you should observe. The unconstrained tree will have ECE ~5–7× higher than the depth-5 tree. Pure leaves in an unbounded tree produce probabilities of exactly 0 or 1 — when a leaf contains only one sample, the model says "100% churn" or "0% churn." Those extreme predictions are dramatically miscalibrated when you test on unseen customers. The depth constraint forces leaves to contain multiple samples, producing smoother, more trustworthy probability estimates. **This is why tree depth is about probability quality, not just overfitting.**

Task 3 — Random forest and feature importances

Implement `build_random_forest(X_train, y_train, n_estimators=100, max_depth=10, class_weight=None, random_state=42)`:

- Train a `RandomForestClassifier` with the given parameters
- Return the fitted model

Note the `max_depth=10` default. Production random forests almost always constrain depth — unbounded forests overfit noise on individual trees (even though ensemble averaging mitigates this somewhat, the constraint makes the model more reliable and interpretable).

Then implement `get_feature_importances(model, feature_names)`:

- Zip feature names with `model.feature_importances_`
- Sort descending by importance
- Return as a dict

Task 4 — class_weight at the default 0.5 threshold

Train a second random forest with `class_weight="balanced"`. Implement `evaluate_recall_at_threshold(model, X_test, y_test, threshold=0.5)`:

- Get `predict_proba(X_test)[:, 1]`
- Threshold at `threshold` to get binary predictions
- Return `recall_score` on the positive class

Compare recall **at the default 0.5 threshold** between default and balanced random forests. Then implement `compute_pr_auc(model, X_test, y_test)` using `average_precision_score(y_test, predict_proba)` and compute PR-AUC for both models.

What you should observe. Recall at 0.5 threshold will roughly double for the balanced model (e.g., 0.16 → 0.40). But **PR-AUC will be approximately equal or even slightly lower for the balanced model**. This is the honest lesson:

> *`class_weight="balanced"` reweights the training loss, which shifts predicted probabilities upward. At the default 0.5 threshold, more samples cross into the positive prediction, so recall at that threshold goes up. The PR curve itself — the model's threshold-independent ranking of customers from most-to-least likely to churn — is largely unchanged. `class_weight` is an operating-point tool, not a ranking-improvement tool.*

Every time you describe the recall improvement in your write-up, you must use the "at the default 0.5 threshold" qualifier.

Task 5 — PR curves and calibration curves

Implement `plot_pr_curves(rf_default, rf_balanced, X_test, y_test, output_path)`:

- Create a figure
- Use `PrecisionRecallDisplay.from_estimator` for each model on the same axes
- Save to `output_path` (e.g., `results/pr_curves.png`) and close the figure

Implement `plot_calibration_curves(rf_default, rf_balanced, X_test, y_test, output_path)`:

- Use `CalibrationDisplay.from_estimator` for each model on the same axes, with `n_bins=10`
- Save to `output_path` (e.g., `results/calibration_curves.png`)

What you should observe on the PR curves. The two curves will look very similar (consistent with PR-AUC being nearly equal). This is the threshold-independent view of class_weight: the model's ranking is basically the same, regardless of loss reweighting.

Task 6 — Tree-vs-linear capability demonstration

Implement `build_logistic_regression(X_train_scaled, y_train, random_state=42)`:

- Train a `LogisticRegression(max_iter=1000, random_state=random_state)` on scaled features
- Return the fitted model

Linear models require their inputs on a common scale. Before calling this function, `main()` does:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Then implement `find_tree_vs_linear_disagreement(rf_model, lr_model, X_test_raw, X_test_scaled, y_test, feature_names, min_diff=0.15)`:

- Get `predict_proba(..., 1)` for the RF on `X_test_raw`
- Get `predict_proba(..., 1)` for the LR on `X_test_scaled`
- Compute `|rf_proba − lr_proba|` for every test sample
- Find the sample with the **maximum** difference (must be ≥ `min_diff`, or return None)
- Return a dict with `sample_idx`, `feature_values` (dict of name → value), `rf_proba`, `lr_proba`, `prob_diff`, and `true_label`

The point of this task. Aggregate metrics (PR-AUC) on this data show trees only slightly ahead of linear models. That's common on any moderate-signal dataset — logistic regression is a strong baseline. But trees are genuinely capable of things linear models cannot express: **feature interactions** (the combined effect of two features), **non-monotonic relationships** (where the same feature can increase or decrease risk depending on the value), and **sharp thresholds** (step-function decision rules). Finding a specific customer where the two models disagree dramatically — and explaining the disagreement in structural terms — is your evidence that trees have capabilities linear models don't, regardless of aggregate metrics.

Task 7 — Write the summary

In your PR description, write:

1. Classification report for default RF vs balanced RF (copy-paste from `classification_report`)
2. Top 5 features by importance (from the RF with `max_depth=10`)
3. PR-AUC values for DT (`max_depth=5`), RF default, and RF balanced
4. ECE values: DT `max_depth=None` vs DT `max_depth=5`
5. **The tree-vs-linear disagreement**: sample index, all feature values, both predicted probabilities, true label, and a 2–3 sentence structural explanation. Example:

   > *"Sample 777 — tenure=36, monthly_charges=20, contract_months=1 (month-to-month), num_support_calls=2. RF predicts P(churn)=0.60; LR predicts P(churn)=0.17. The random forest captured the interaction: customers with several support calls on a month-to-month plan are at elevated risk regardless of tenure. Logistic regression can only weight each feature linearly, so it combines the long-tenure protection with the modest individual risk of month-to-month and arrives at a much lower probability. The tree's ability to split on `contract_months <= 1` and then further split on `num_support_calls > 2` inside that branch captures a pattern the linear model structurally can't express."*

6. Brief comparison to Week A logistic regression (~3 sentences). **Use the "at the default 0.5 threshold" qualifier** every time you describe the class_weight effect on recall.

Submission

Open a pull request from `lab-5b-tree-models` to `main`. Your PR must include:

1. Completed `lab_trees.py` that runs end-to-end
2. All output files in `results/`: `decision_tree.png`, `pr_curves.png`, `calibration_curves.png`
3. PR description with items 1–6 from Task 7
4. Paste your PR URL into TalentLMS → Module 5 Week B → Applied Lab to submit this assignment.

Challenge Extensions

These are optional extensions for learners who finish the base requirements. Not graded, but they'll deepen your understanding.

Tier 1 — Threshold tuning

The default 0.5 threshold is rarely optimal on imbalanced problems. Using `predict_proba` from the balanced random forest, sweep thresholds from 0.1 to 0.9 in steps of 0.05. For each threshold, compute precision, recall, and F1. Plot all three against threshold on one figure. Identify:

- The threshold that maximizes F1
- The threshold that achieves at least 80% recall

Write a brief note: if Petra Telecom can contact 200 customers per month for retention offers, which threshold would you recommend? Reference both the cost side (false positives = wasted effort) and the opportunity side (false negatives = lost customers).

Save to `results/threshold_sweep.png`.

Tier 2 — Permutation importance

The default `feature_importances_` in random forests (mean decrease in impurity) is biased toward high-cardinality features. Permutation importance is a more reliable alternative.

- Use `sklearn.inspection.permutation_importance` on the balanced random forest on the test set
- Compare the permutation-importance ranking to the MDI ranking from Task 3
- Create a side-by-side bar chart comparing MDI and permutation importance for the top 10 features
- Write a paragraph explaining when and why the two methods disagree

Save to `results/permutation_vs_mdi.png`.

Tier 3 — Custom voting ensemble

scikit-learn provides `VotingClassifier`, but building your own teaches you what's inside.

Implement a custom ensemble class that:

- Accepts a list of fitted sklearn classifiers
- Implements `predict(X)` via majority voting across all classifiers
- Implements `predict_proba(X)` by averaging the `predict_proba` outputs
- Handles the case where classifiers disagree on class ordering (check `.classes_` attributes)

Combine the Week A logistic regression, your balanced decision tree, and your balanced random forest into your custom ensemble. Evaluate with classification report + PR-AUC. Compare to each individual model. Does the ensemble outperform the best individual model? When would you expect it to?

The implementation should follow sklearn conventions (`.fit()`, `.predict()`, `.predict_proba()`) but does not need to inherit from sklearn base classes.